Chapter 10: Elementary Data Structures

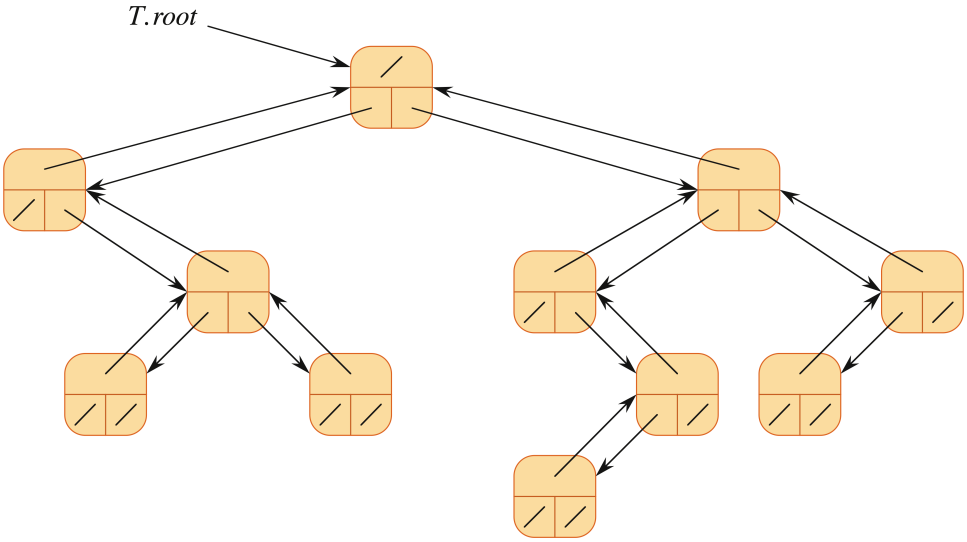
Section 10.3: Representing Rooted Trees

- Trees are composed by tree nodes.
- Each tree node has a key field and some other pointer fields pointing to other nodes. Number of pointer fields in a tree node may be different for different types of trees.
- A tree T has an attribute $T.root$: a pointer to the root of the tree.

Binary Trees

For each node x, there are 3 pointer fields and one data field

- $x.p$ is a pointer to x's parent.
- $x.left$ is a pointer to x's left child.
- $x.right$ is a pointer to x's right child.
- $x.data$ is a pointer to x's satellite data



```java
public class BinaryTree<T> {
    private Node<T> root;

    // Note that the generic variable for Node must be different than the one
    // for BinaryTree, otherwise it will cause a compile error.
    public class Node<S> {
        private Node<S> parent;
        private Node<S> left;
        private Node<S> right;
        private S data; //arbitrary data
        //...
    }
    //...
}
```

Rooted Tree with Bounded Branches

We can represent a general tree that has a bounded number of branches by using an array of pointers. Let the bound be r (also know as the degree). A node in such a tree will have the following fields:

- *x.p* is a pointer to x's parent.
- *x.child[1:r]* is a pointer to x's children, up to r of them.
- *x.data* is a pointer to x's satellite data

In general, this would be space inefficient as most of the child pointers will be NIL. But it provides an easy and fast way to access the i child of any node.

Assume that the array of child pointers starts at 0. We will denote r using the variable `degree` in the code below.

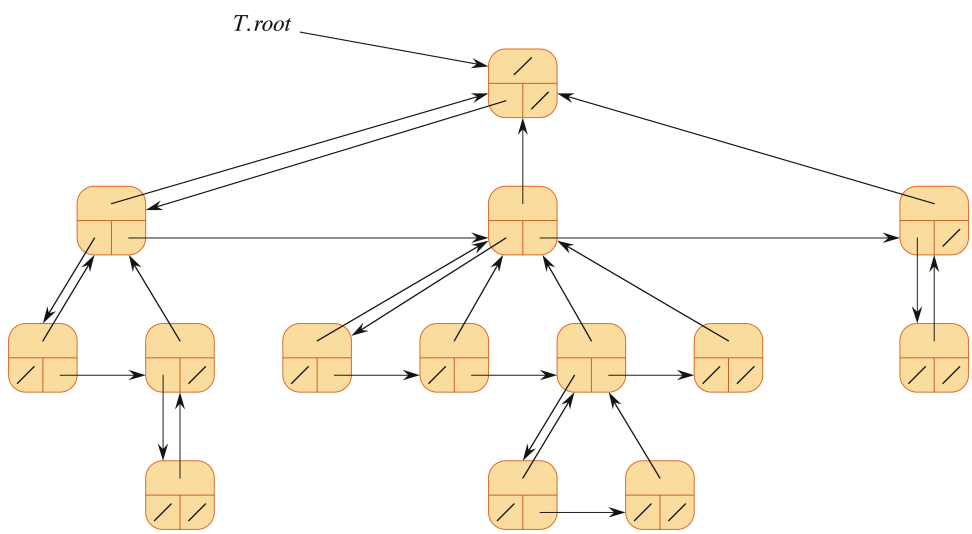
```java
public class ArbitraryTree<T> {
    private Node<T> root;
    private int degree;

    public Tree(int degree) {
        this.degree = degree;
    }
    public class Node<S> {
        private Node<S> parent;
        private Node<S> child[];
        private S data; //arbitrary data

        //...
    }
    //...
}
```

Rooted Trees with Unbounded Branches

- Each node can have any number of children.
- Using left-child, right-sibling representation allows us to represent an arbitrary tree using only three pointers per node.
- Three pointer fields for each node x.
  - $x.p$ is a pointer to x's parent.
  - $x.left$ is a pointer to x's left-most child.
  - $x.right$ is a pointer to the sibling of x immediately to the right.



- Write the Java class declaration for the left-child right-sibling representation of an arbitrary tree.

Recommended Exercises

:

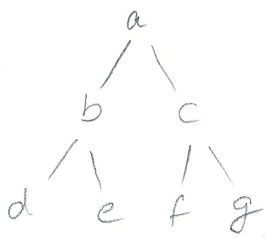
- **Exercise 10.3-1**. Left as an exercise for the reader.
- **Inclass - Exercise 10.3-2**. Let us develop a simple recursive solution for printing key (or data) of each node in a binary tree.

```
PrintAll (root)
// Print the data of each node in the subtree at root
```

- **Exercise 10.3-4**. Write a simple $O(n)$ recursive solution for printing key (or data) of each node in a tree with unbounded branches, represented using the left-child right-sibling representation.
  enditemize

Tree Traversals

- A tree traversal is a systematic way of visiting all the nodes in a tree exactly once. Tree traversals are used in many applications. For example, we can use a tree traversal to print the key of each node in a tree (like we did above).
- We will discuss three types of tree traversals: preorder, inorder, and postorder.
- For a binary tree, we can define the three types of traversals as follows:
  - **Preorder**: Visit the root, then recursively visit the left subtree, and finally recursively visit the right subtree.
  - **Inorder**: Recursively visit the left subtree, then visit the root, and finally recursively visit the right subtree.
  - **Postorder**: Recursively visit the left subtree, then recursively visit the right subtree, and finally visit the root.
- For a tree with unbounded branches, we can define the three types of traversals as follows:
  - **Preorder**: Visit the root, then recursively visit each child from left to right.
  - **Inorder**: Recursively visit the first $\lceil n/2 \rceil$ children from left to right, then visit the root, and finally recursively visit the remaining $\lfloor n/2 \rfloor$ children from left to right.
  - **Postorder**: Recursively visit each child from left to right, and finally visit the root.



inorder traversal

d → b → e → a → f → c → g

preorder traversal

a → b → d → e → c → f → g

postorder traversal

d → e → b → f → g → c → a

- **Recommended Exercises**:
  - Write a simple $O(n)$ recursive solution for printing key (or data) of each node in a binary tree using preorder, inorder, and postorder traversals.
  - Show the output from your three procedures for a binary tree of your choice.